

Tiny Tapeout GPS Accelerator

It's counters all the way down

Presentation Overview

- Why TinyTapeout
- Background
- Design Methodology
- System Breakdown
- Testing
- TT with VHDL
- Questions

Why Tiny Tapeout

They did it in the 80s

- GPS is pretty well documented
- GPS can be received with a very low dynamic range (single bit samples)
- Scalable number of channels and features to fit in TT tiles
- It's might be useful to someone (rocketry?)
- Discrete logic designs online

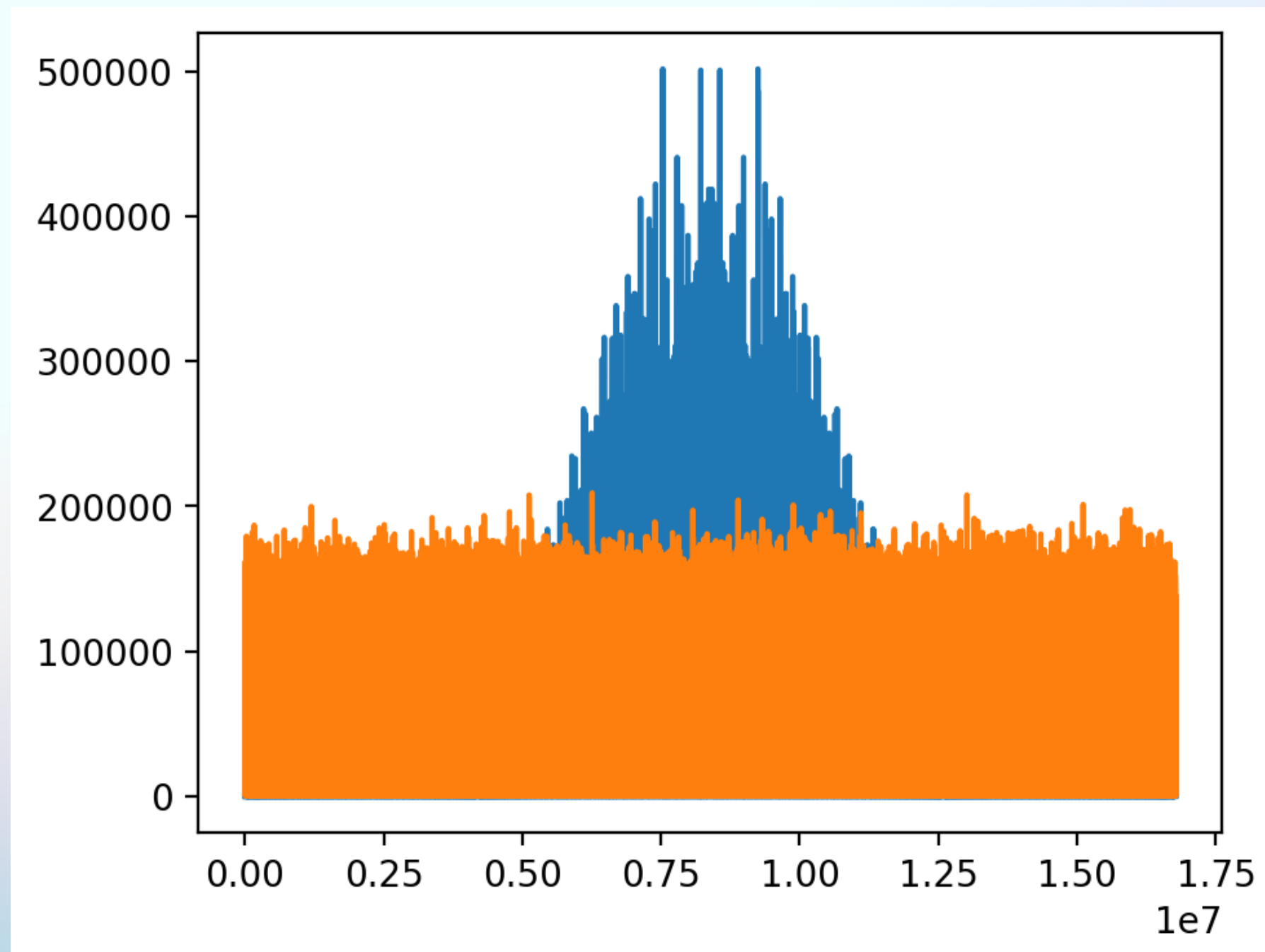
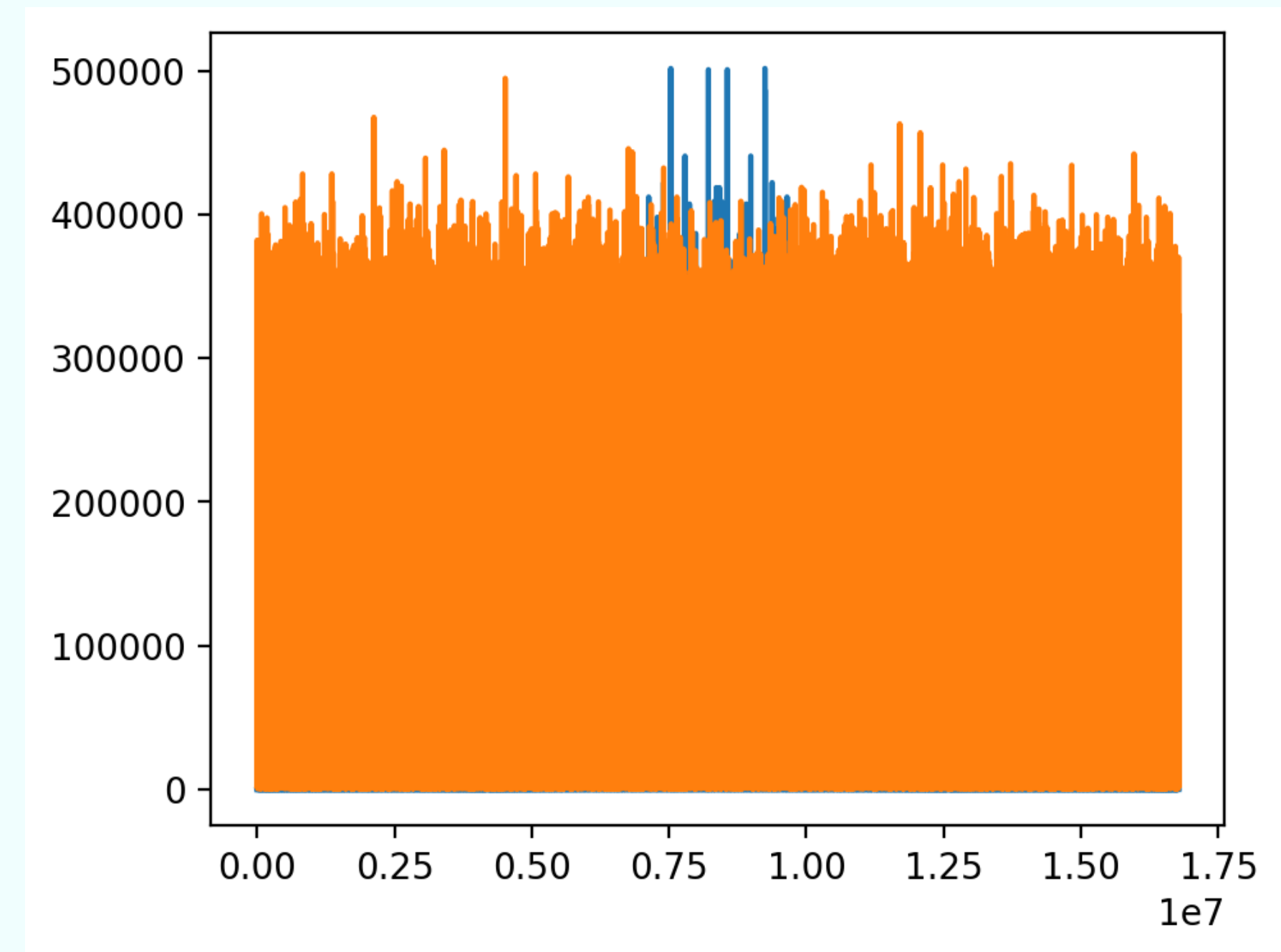


Background

- Low power at surface of earth so Direct Sequence Spread Spectrum used to provide Processing Gain
- GPS uses CDMA to separate the overlapping satellite transmissions
- Signal powers are about the same (and below the thermal noise floor) so low dynamic range receivers can be used
- This design handles the “digital” correlation and tracking (still needs RF Front end and CPU based logic)

A Software-Defined GPS and Galileo Receiver: A Single-Frequency Approach

Kai Borre, Aalborg University; Dennis M. Akos, University of Colorado; Nicolaj Bertelsen Peter Rinder Søren Holdt Jensen, Aalborg University
Birkhäuser, 2007
ISBN: 0-8176-4390-7; Language: English



Design Methodology

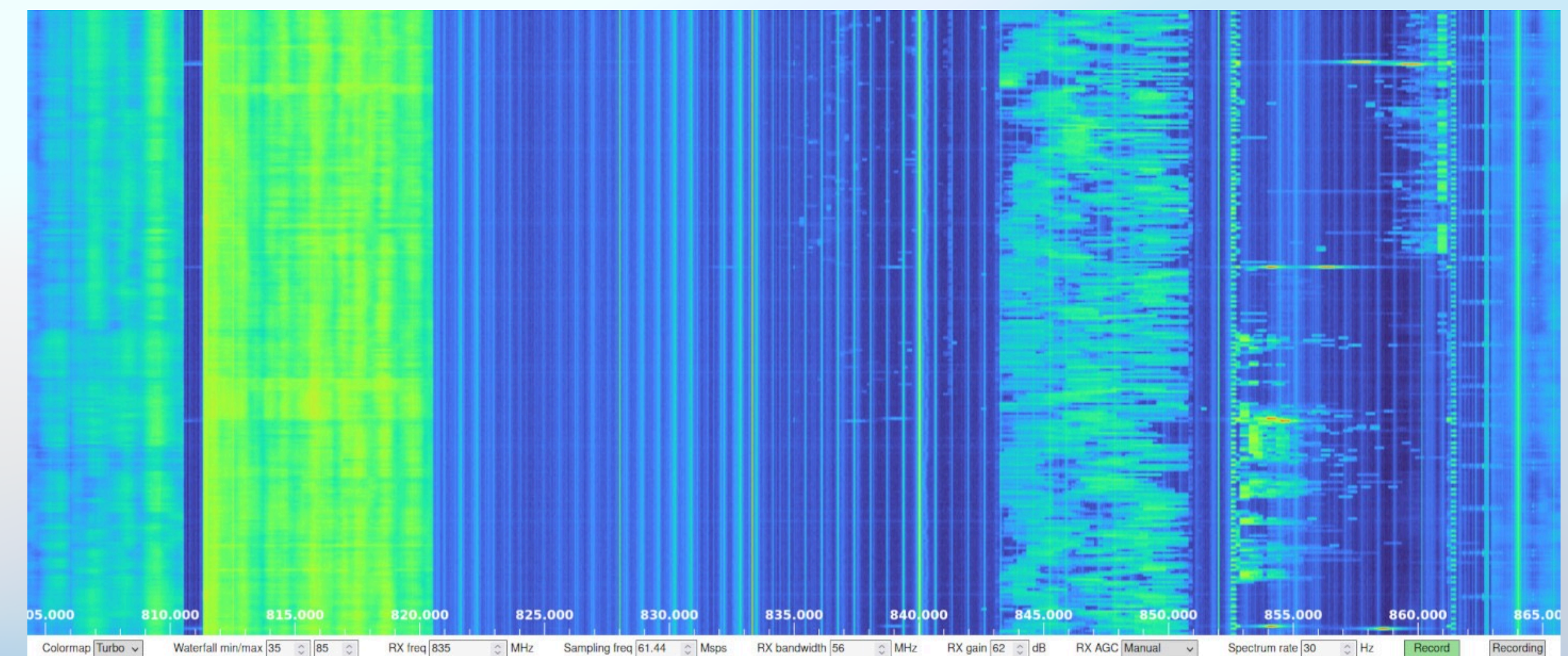
- Jupyter notebooks for algorithm development and test
- VHDL implementation
 - Flexible bit widths via use of generics
 - generates for channel based flexible implementation
- SDC for timing constraint - helpful discussion on Discord
- Cocotb based testbenching - including SPI Master configuration and readback/logic
- Maia/GNSSSDR based real validation - marking your own homework
 - In absence of MAX2771 availability ADALM-PLUTO.+ LeoBodmar + Active antenna used
 - Maia-SDR used to record complex 8bit samples at 4.092MHz (4x oversampled)
 - GNSSSDR Docker can process 2 bit samples, convert 8bit signed to single bit and di-bit representations

System Breakdown

- Inputs: I/Q single bit streams + sample clock
- Outputs: Timing Interrupt
- IO: SPI based interface for config and readback of registers
 - 32 bit word, 5 bit address, R/W, 16 bit data, 8-b CDC delays
- 1 Automated Acquisition engine
 - Search 1023 possible code phase offsets across a range of frequencies for a single SV
- 3 Tracking Channels
 - Independently track Early/Mid/Late code phases for given time, frequency and SV
 - Dedicated NCO and Gold Code Generator

Testing

- Jupyter generate synthetic data set with known perturbations for early testing:
 - Time (code phase)
 - Frequency
 - Combination of SV's
- GPS Analog Front end hard to purchase, used Analog Devices PlutoSDR to record
 - Converted to single bit samples
 - Compared to GNSS-SDR results
- Jupyter confirmed processing with real recordings



TinyTapeout with VHDL

- There is VHDL repos available - ended up just using the GF verilog one
- Stock tinytapeout attempts to transpile VHDL to Verilog if Language is set to VHDL in info.yaml
 - Not very good if you use anything complex language wise like generics
- LibreLane underneath TinyTapeout handles VHDL just fine with the VHDLClassic flow (update config.json)
- Make sure you use VHDL-2008 switches to GHDL if you want “modern” features
- Tweaks to the Github actions to skip the “Linter” stages as it’s expecting verilog messages
- Testbenches in cocotb means you don’t have to write different RTL and Gate level tests
 - iverilog used for gate level tb even though I don’t know how to write verilog, all just works

Questions